

Sorting Algorithm Benchmarking in Python: Effects of Input Size and Ordering

Shreyash

CSC506: Design and Analysis of Algorithms

Colorado State University Global

October 2026

Abstract

This project evaluated Bubble Sort, Selection Sort, Insertion Sort, and Merge Sort across random, sorted, reverse-sorted, and partially sorted integer lists with 1,000, 5,000, 10,000, and 50,000 elements. Each scenario was measured once in Python 3.14.4 on Windows 11 using `time.perf_counter()`, and every result was validated against Python's independently computed `sorted(original)` output.

The results supported the expected algorithmic patterns: Merge Sort scaled best on random, reverse-sorted, and partially sorted inputs, while adaptive quadratic algorithms were strongest on already sorted inputs. Because the benchmark used one recorded trial per scenario, the findings should be read as transparent project evidence rather than statistically generalizable performance claims.

Introduction

Sorting algorithms connect theoretical complexity with observed runtime behavior. Cormen et al. (2022) present sorting as a core topic in algorithm analysis because different strategies solve the same problem with different time and space costs.

SortBench compared four manually implemented algorithms under controlled project conditions. Bubble Sort and Insertion Sort can exploit already ordered data, Selection Sort performs predictable quadratic scans, and Merge Sort uses divide and conquer to provide $O(n \log n)$ behavior at the cost of auxiliary storage.

Method

The benchmark used four dataset types and four sizes, producing 64 scenarios. Each size used deterministic seeds, and all orderings for that size shared the same multiset of values. The timer measured only the sorting call; generation, copying, validation, and CSV output were excluded.

Python's `time.perf_counter()` was selected because the official documentation describes it as a high-resolution performance counter for measuring short durations (Python Software Foundation 2026a). Python's `sorted` function was used only as a correctness oracle, not as a benchmark target.

Results

Merge Sort was fastest in all random, reverse-sorted, and partially sorted groups. Sorted input changed the ranking: Bubble Sort won at 1,000, 5,000, and 50,000 elements, while Insertion Sort won at 10,000 elements. Selection Sort remained slow on sorted input because it continued scanning each unsorted suffix.

Table 1

Execution Time for 50,000 Elements, in Seconds

Ordering	Bubble	Selection	Insertion	Merge
Random	61.1792	23.6449	27.0069	0.0731
Sorted	0.0013	29.2821	0.0026	0.0695
Reverse	79.5163	28.0274	52.9517	0.0631
Partial	40.8929	26.2688	3.2475	0.0819

Analysis Figures

Figure 1
Execution time by dataset size.

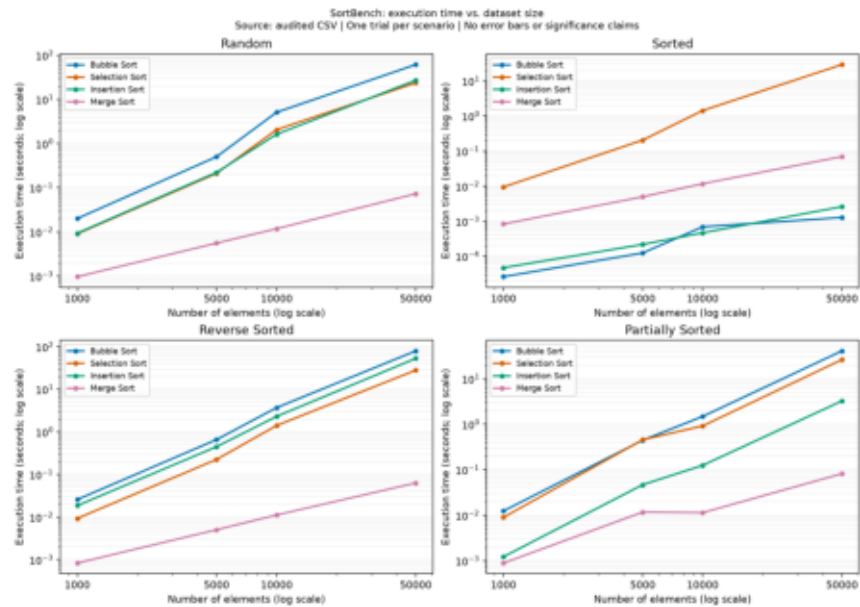
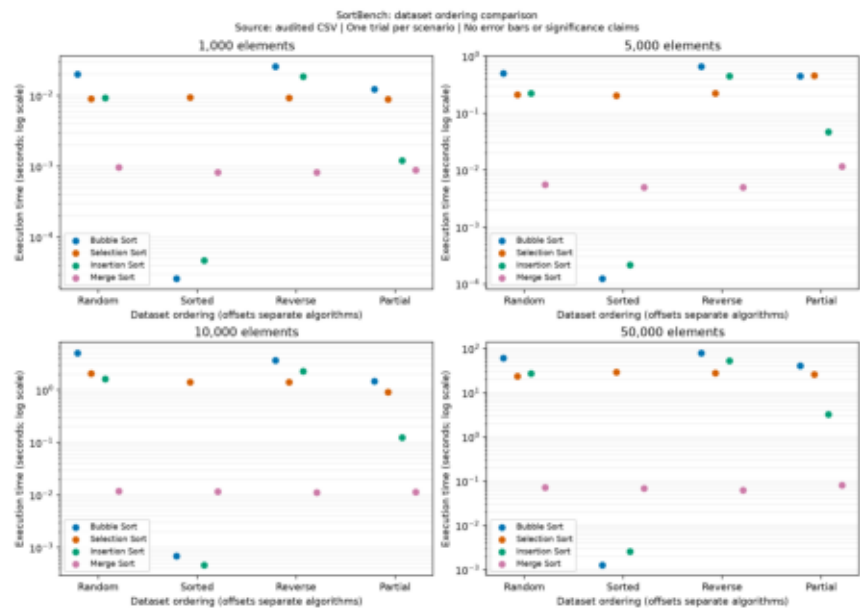


Figure 2
Algorithm comparison by dataset ordering.



Dataset Figures

Figure 3
Random dataset performance.

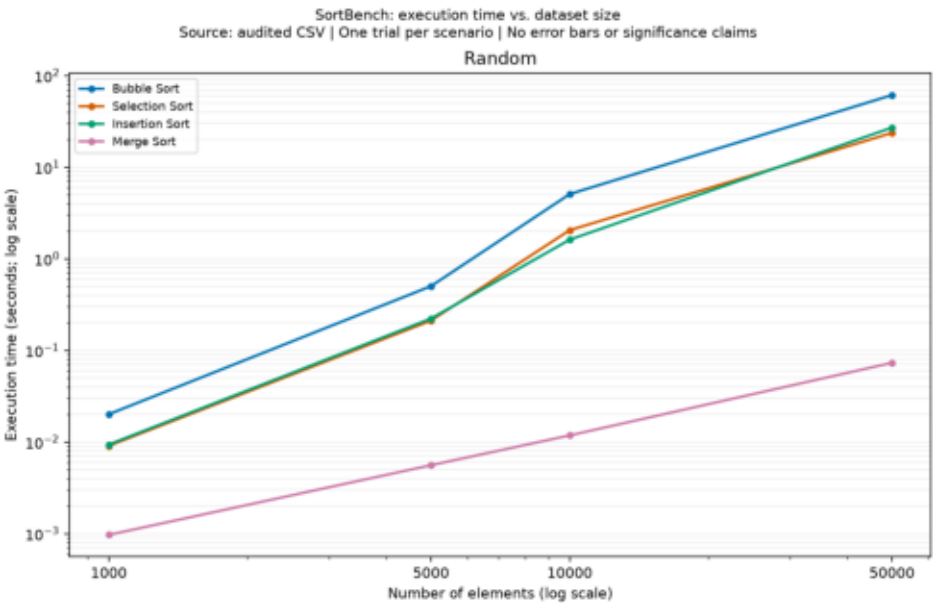
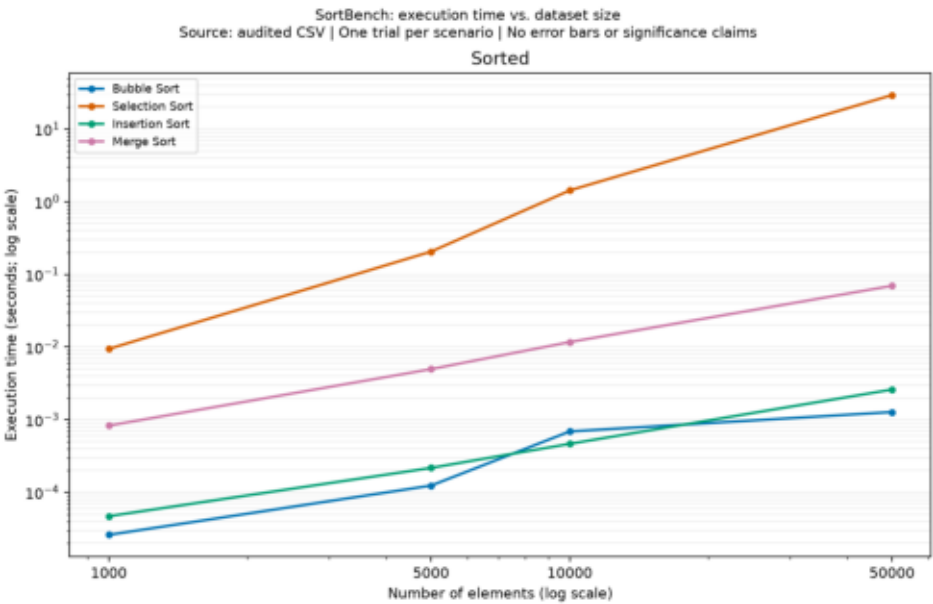


Figure 4
Sorted dataset performance.



Dataset Figures

Figure 5
Reverse-sorted dataset performance.

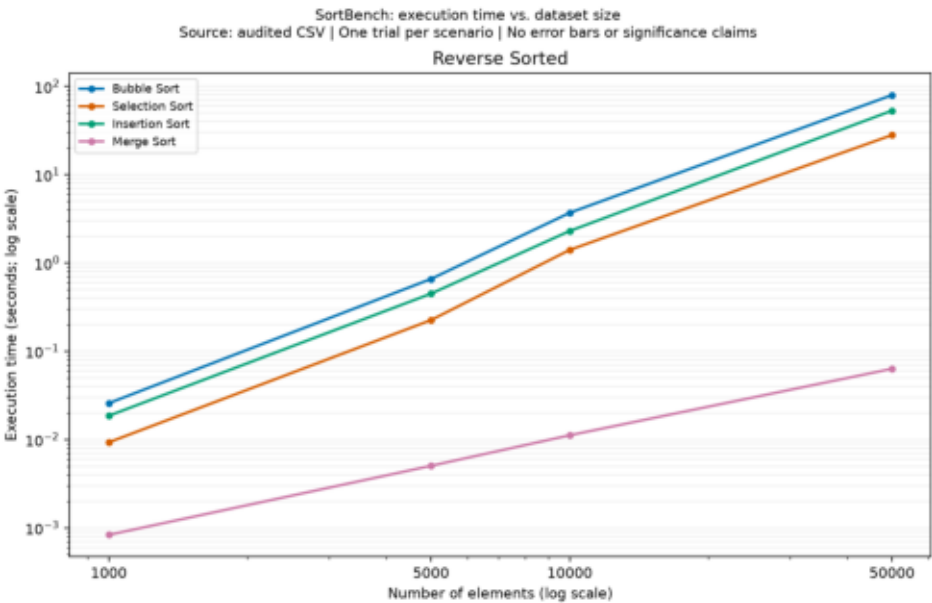
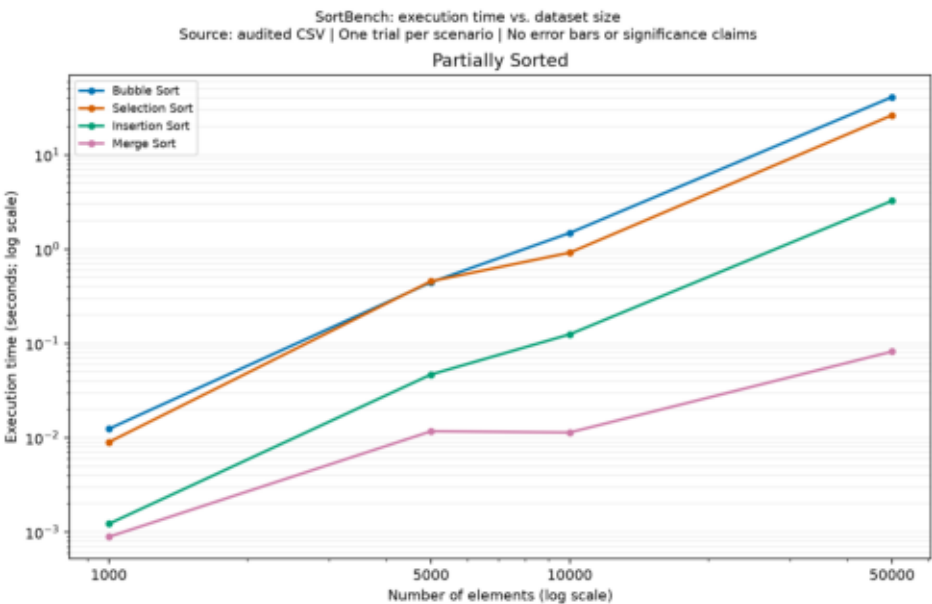


Figure 6
Partially sorted dataset performance.



Discussion

The measured growth rates were consistent with the distinction between $O(n \log n)$ and $O(n^2)$. On random input, increasing the size from 1,000 to 50,000 elements increased Merge Sort runtime by about 75.5 times, while the quadratic algorithms increased by roughly 2,606 to 3,047 times.

Input ordering mattered most for Bubble Sort and Insertion Sort. On sorted input, both algorithms behaved like adaptive algorithms and avoided the large costs seen on random or reverse-sorted data. Python's Sorting HOWTO similarly notes that Python's Timsort can take advantage of existing order, although SortBench measured the course algorithms rather than Python's built-in sort (Python Software Foundation, 2026b).

Recommendations and Limitations

For random, reverse-sorted, or partially sorted integer lists in the tested size range, Merge Sort is the best time-based choice when $O(n)$ extra memory is acceptable. For already sorted data, Bubble Sort or Insertion Sort is more appropriate. The main limitation is that each scenario has one recorded trial, so future work should add repeated trials, randomized execution order, multiple seeds, and memory profiling.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). The MIT Press. <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>

Knuth, D. E. (1998). The art of computer programming: Volume 3: Sorting and searching (2nd ed.). Addison-Wesley.

Python Software Foundation. (2026a). time - Time access and conversions. Python 3.14 documentation. <https://docs.python.org/3/library/time.html>

Python Software Foundation. (2026b). Sorting techniques. Python 3.14 documentation. <https://docs.python.org/3/howto/sorting.html>